Documento de Análisis y Diseño

de

BioSpace

Fecha	02-10-2026
Versión	0.1
Integrantes del grupo	Felipe Urbina Ian Jeria Felix Barrera Lukas Astudillo

Introducción Diagnóstico e identificación del problema, desafío u oportunidad

En los espacios domésticos modernos, el cuidado de las plantas de interior se ha convertido en una tarea compleja y, a menudo, en un verdadero desafío cotidiano. Esta labor suele verse dificultada por la falta de tiempo consecuencia de horarios laborales exigentes o por condiciones de iluminación desfavorables, caracterizadas por la escasez o incluso la ausencia total de luz solar directa.

Tradicionalmente, el riego y la ubicación de las plantas se han basado en la experiencia, guiados por la intuición o una simple evaluación visual (un enfoque informal conocido popularmente como cuidado "al ojo"). Esta falta de métodos estructurados conduce a extremos perjudiciales para la salud de la vegetación: por un lado, existe el riesgo constante de un riego excesivo que puede asfixiar el sistema radicular y favorecer la proliferación de hongos; por otro, surge la amenaza de una deshidratación severa que provoca la muerte prematura de la planta simplemente por haber olvidado regarla.

A este factor humano se suma una variable física fundamental: la interacción entre el diseño espacial de la vivienda y la dinámica de evaporación del agua en el sustrato. Esta combinación de humedad y temperatura genera microclimas interiores muy diversos y difíciles de comprender de manera intuitiva, lo que complica considerablemente el establecimiento de una rutina de riego constante y adaptada a las necesidades específicas, especialmente en el caso de especies vegetales sensibles.

Para abordar esta problemática de manera eficaz —y sin recurrir a costosos sistemas de hardware externo— se desarrolló el proyecto **BioSpace**. Esta solución consiste en una aplicación interactiva programada en C++, diseñada bajo un enfoque orientado a objetos e integrada con una gestión precisa y manual de la memoria dinámica en el *heap*. El sistema modeliza matemáticamente la compleja relación multifactorial entre la dinámica de la humedad del suelo y la radiación solar en las distintas zonas del hogar. Mediante la simulación de estados de sequía en intervalos de tiempo discretos y el uso eficiente de punteros y estructuras de datos dinámicas, el sistema optimiza la ubicación de cada planta y proporciona instrucciones de riego precisas y proactivas, elevando así el cuidado vegetal a un nivel de precisión tecnológica avanzado.

1.1 Diagnóstico e identificación del problema, desafío u oportunidad

En el ámbito del cuidado de la vegetación doméstica, un alto porcentaje de las plantas de interior perece prematuramente debido a dos causas principales: la pudrición radicular por sobre-riego y la deshidratación severa por falta de humedad en el sustrato o exceso de radiación. La dificultad central radica en que los requerimientos hídricos no son estáticos; varían drásticamente dependiendo del microclima específico de cada habitación. Por ejemplo, una maceta ubicada junto a una ventana orientada al sur experimenta una mayor tasa de radiación solar y una evaporación acelerada, mientras que esa misma planta situada en el baño o en una habitación interior retiene la humedad en el suelo durante períodos prolongados.

Actualmente, la mayoría de las personas recurren a calendarios fijos de riego (como regar cada 5 días) o al ensayo y error, ignorando por completo la física del microclima local. Aunque existen sensores IoT de humedad en el mercado, estos requieren el uso de baterías y un mantenimiento constante que, para un usuario común, representa un costo económico extra y una molestia operativa.

Esta situación genera una oportunidad tecnológica y computacional invaluable para desarrollar un modelo de software determinista capaz de simular el comportamiento termodinámico del agua en el sustrato y la exposición solar teórica según la orientación geográfica (Norte, Sur, Este, Oeste). El desafío radica en traducir la interacción entre las habitaciones, las ventanas y las especies vegetales en un sistema ágil, eficiente y libre de fallos de memoria que pueda ejecutarse de manera fluida.

1.2 Propuesta de solución

Para abordar el problema del cuidado ineficiente de las plantas domésticas, se propone desarrollar **BioSpace**, una aplicación de escritorio local desarrollada en C++ e integrada con el framework **Qt** para construir una interfaz gráfica de usuario intuitiva. El sistema registra y organiza los datos del usuario, clasificando y monitoreando el estado de las plantas mediante retroalimentación visual para mitigar el descuido humano, e incorpora un sistema de notificaciones basado en las condiciones ideales que cada especie necesita a lo largo del día.

En la actualidad existen diversas aplicaciones orientadas al cuidado de plantas; sin embargo, **BioSpace** destaca frente a estas alternativas al ofrecer un sistema de inventario persistente y la capacidad de gestionar múltiples ejemplares de forma personalizada dentro de una misma plataforma. Las soluciones dominantes en el mercado actual suelen basarse en el análisis fotográfico individual de la planta, lo que obliga al usuario a tomar fotografías diarias de cada maceta de manera separada —incrementando el riesgo de omisión— y limita sus funciones avanzadas a modelos de suscripción de pago.

En contraste, **BioSpace** se orienta a usuarios que buscan una herramienta de gestión rápida, ligera, privada y completamente personalizada en su computador personal. Al ser gratuita y no depender de análisis fotográficos diarios ni de suscripciones, se posiciona como una alternativa visualmente atractiva, precisa y altamente competitiva frente a las opciones comerciales tradicionales.

2.0 Requisitos del Sistema

[Esta sección describe los requisitos funcionales del sistema, sus interfaces externas y el perfil de los usuarios esperados para el sistema.]

2.1 Requisitos Funcionales

- **RF1 - Registro de planta:** El sistema permite al usuario registrar ciertos tipos planta ingresando de manera obligatoria su nombre y especie (si el usuario la desconoce, se aplicarán condiciones generales de cuidado, con valores promedios para la humedad, sol, cantidad de tierra, etc.), ofreciendo de manera *opcional* la posibilidad de seleccionar una fotografía de los archivos locales para referenciarla.

- **RF2 - Visualización:** El sistema debe mostrar el inventario en la interfaz principal. Para cada planta ingresada, se exhibirá la fotografía de la planta (o su nombre) en un cuadro. acompañado de dicho cuadro, el nivel actual de hidratación de forma gráfica.

- **RF3 - Simulación hídrica:** El sistema debe calcular de manera dinámica los niveles de agua, tasa de de evaporación en función de la radiación solar, esto en función de las necesidades específicas de cada planta y el tiempo transcurridos así mismo debe evaluar si el nivel de agua si desciende por debajo del umbral crítico emitir alertas de riego en consola.

- **RF4 - Acción hídrica:** El sistema debe implementar un botón interactivo que, al ser seleccionado, restaure los niveles hídricos necesarios para una planta específica. También incluirá un botón general para rellenar los niveles de todo el inventario simultáneamente.

- **RF5 - Persistencia de datos:** El sistema debe guardar de manera automática el inventario de plantas, para que las interacciones entre distintas sesiones vaya acorde a lo que el usuario ingresa y necesita aparte El software debe permitir guardar el historial acumulado de humedad, días transcurridos y decisiones sobre el estado de la planta.

2.2 Requisitos no funcionales

- **RNF1 - Interfaz:** El sistema debe operar a través de la interfaz gráfica de usuario, es decir, debe prescindir de la interacción con la consola de comandos para facilitar la experiencia del usuario.

- **RNF2 - Privacidad:** El software debe gestionar y procesar la totalidad de la información (datos del inventario, parámetros de las estancias y archivos de fotografías) exclusivamente en el almacenamiento local del equipo, garantizando la privacidad de la información sin transmitir datos a servidores de terceros.

- **RNF3 - Operatividad Offline:** Todas las funcionalidades principales del sistema incluyendo la simulación hídrica, la visualización del inventario y el control de riego deben estar totalmente operativas sin necesidad de conexión a internet.

- **RNF4 - (posible bot telegram):** Como extensión independiente y opcional, el sistema podrá integrar un módulo de mensajería mediante un bot de Telegram. Este módulo enviará alertas remotas de riego al dispositivo móvil del usuario únicamente cuando el equipo disponga de conexión a red, sin comprometer la autonomía ni la operatividad offline del sistema base.

2.3 Requisitos de Ambiente

Esta sección describe el hardware y software relevante para la solución.

2.3.1 Hardware de Desarrollo

Arquitectura del Procesador: Computadores personales con arquitectura de procesador x86 o x64.

Memoria RAM: Mínimo 4 GB de memoria RAM (se recomiendan 8 GB para optimizar los tiempos de compilación de proyectos con el framework Qt y herramientas de análisis).

Almacenamiento: Mínimo 2 GB de espacio libre en disco para el código fuente, la biblioteca de gráficos (Qt), los assets de imágenes y los archivos planos de persistencia.

2.3.2 Software de Desarrollo

Sistema Operativo: Compatibilidad multiplataforma con distribuciones de Linux (ej. Ubuntu, Debian) y entornos Microsoft Windows (Windows 10/11).

Lenguaje de Programación: **C++** (estándar C++17 o superior), seleccionado por su alto rendimiento como lenguaje compilado y su control eficiente en la gestión explícita de memoria

Framework de Interfaz Gráfica: **Qt Framework** (Qt 5 / Qt 6) y **Qt Creator** como entorno de desarrollo integrado (IDE) para el diseño de componentes visuales (requiere instalación previa de las librerías Qt Widgets o QML).

Compilador y Herramientas de Auditoría:

Compilador nativo: g++ (GCC) en Linux o msvc / minGW en Windows.

Herramienta de análisis de memoria: **Valgrind** (leak-check=full)) en entorno Linux para certificar la liberación de recursos en el *heap* y evitar fugas de memoria

3 Planificación del Proyecto

Se actualiza con la información de cada bitácora y se mantienen versiones del documento completo asociado a cada bitácora (desarrollo iterativo).

3.1 Objetivo General (Goal)

Desarrollar una aplicación de escritorio local en C++ y Qt que sea un complemento para los usuarios que tienen plantas, con el fin de gestionar y monitorear sus necesidades y cuidados, utilizando una arquitectura de programación orientada a objetos (POO) para simular en base al tiempo tanto su situación hídrica como lumínica, almacenando las especificaciones de cada planta para que los datos persistan en cada sesión.

3.2 Objetivos Específicos

SO1 Desarrollar el motor de simulación hídrica y acciones de riego: Programar los algoritmos de cálculo dinámico de evaporación basados en la radiación solar y el tiempo transcurrido, e implementar las funciones de notificación de alertas de sequía y restauración de humedad (riego individual y global).

SO2 Diseñar e implementar la arquitectura modular orientada a objetos e interfaz gráfica: Construir la estructura de clases del dominio (Planta, habitación, inventario, simulador) y desarrollar la GUI mediante el framework Qt (QGraphicsview, QWidget) para permitir la visualización gráfica del inventario y los niveles de hidratación.

SO3 Administrar y auditar la memoria dinámica en el heap: Gestionar el ciclo de vida de los objetos e inventarios mediante asignación dinámica (new/delete,new[], delete[]), utilizando punteros y referencias de forma eficiente, y certificar la ausencia total de **fugas de memoria (memory leaks)** y **punteros colgados (dangling pointers)** mediante la herramienta **Valgrind** (leak-check=full)

SO4 Implementar el sistema de persistencia local, procesamiento de datos y notificaciones: Configurar el almacenamiento automático del inventario y la exportación de archivos CSV para asegurar la continuidad entre sesiones , permitiendo además procesar el historial acumulado como material de apoyo visual para la evaluación del proyecto. Finalmente, integrar un módulo de mensajería mediante un bot de Telegram cuando el programa estime necesario.

3.3 Actividades y Milestones

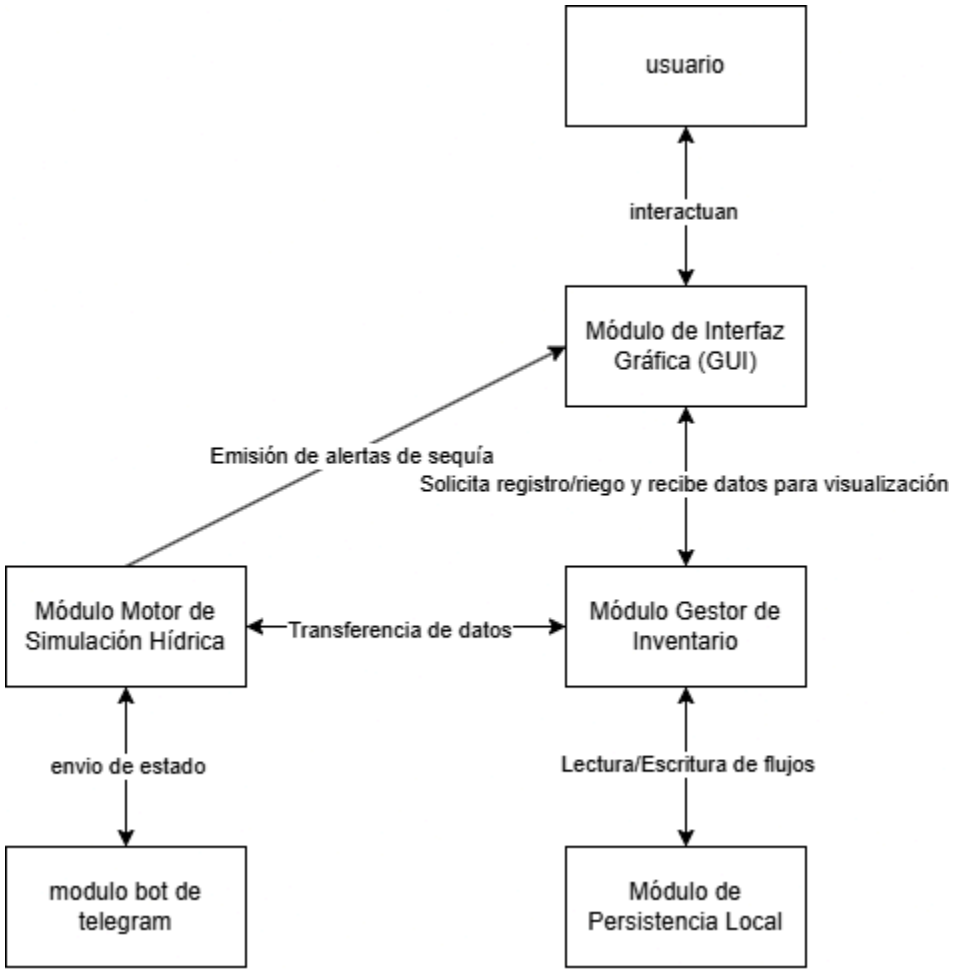
Hitos, objetivos específicos, actividades	Descripción	Responsable(s) (Distribución)	Comienzo	Fin	% Esperado	% Logrado
H0	Hito 0: Entrega Documento Análisis y Diseño	Equipo(100%)	30/09/2026	02/10/26	100%	100%
SO1	Motor de simulación hídrica y acciones de riego	Equipo (100%)	05/10/2026	06/11/2026	0%	0%
A1.1	Diseño matemático de algoritmos y fórmulas de evaporación	F.Barrera(50%) L.Astudillo(30%), F.Urbina(10%), I. Jeria (10%)	05/10/2026	19/10/26	0%	0%
A1.2	Lógica de notificación y restauración de humedad (riego)	F.Urbina(50%), I.Jeria (30%), F. Barrera (10%), L.Astudillo (10%)	26/10/2026	06/11/26	0%	0%
SO2	Arquitectura modular POO e interfaz gráfica Qt	Equipo (100%)	19/10/2026	13/11/26	0%	0%
A2.1	Programar estructura de clases (Planta, Inventario) post-clases	L.Astudillo(50%), F. Urbina (30%), F. Barrera (10%), I. Jeria (10%)	19/10/2026	30/10/26	0%	0%

A2.2	**Desarrollar GUI para visualización (QWidget/QGraphicsView)**	**F. Barrera (50%), I. Jeria (30%), L.Astudillo(10%), F. Urbina (10%)**	**02/11/2026**	**13/11/26**	**0%**	**0%**
H1	**Hito 1: Entrega Prototipo Funcional**	**F. Urbina (25%), L.Astudillo(25%), F. Barrera (25%), I. Jeria (25%)**	**13/11/2026**	**13/11/26**	**0%**	**0%**
SO3	**Administrar y auditar la memoria dinámica en el heap**	**Equipo (100%)**	**09/11/2026**	**25/11/26**	**0%**	**0%**
A3.1	**Gestión de ciclo de vida (new/delete, punteros, referencias)**	**I. Jeria (50%), F. Barrera (30%), L.Astudillo(10%), F. Urbina (10%)**	**09/11/2026**	**18/11/26**	**0%**	**0%**
A3.2	**Certificar ausencia total de memory leaks con Valgrind**	**F.Urbina(50%), I. Jeria (30%), F.Barrera(10%), L.Astudillo (10%)**	**18/11/2026**	**25/11/26**	**0%**	**0%**
SO4	**Persistencia local, procesamiento de datos y API Telegram**	**Equipo (100%)**	**16/11/2026**	**02/12/26**	**0%**	**0%**
A4.1	**Almacenamiento CSV e integración**	**L.Astudillo(50%), F. Urbina (30%) I. Jeria (10%),**	**16/11/2026**	**25/11/26**	**0%**	**0%**

	de alertas con Bot Telegram	**F. Barrera (10%)**				
A4.2	**Procesamiento de historial como material de apoyo visual**	**L.Astudillo(50%), F. Barrera(30%), F. Urbina(10%), I. Jeria (10%)**	**25/11/2026**	**02/12/26**	**0%**	**0%**
H2	**Hito 2: Entrega Final Sistema Integrado**	**L.Astudillo(25%), F. Urbina (25%), F. Barrera (25%), I. Jeria (25%)**	**02/12/2026**	**02/12/26**	**0%**	**0%**

Diseño de Arquitectura del Sistema

Diagrama de Arquitectura



Enumeración de Módulos

1° Módulo de interfaz gráfica solicita acciones de registro o de riego y recibe sus datos para datos para su visualización. También es el destinatario de la emisión de alertas de sequía.

2° Módulo de Gestor de inventario Procesa las solicitudes de la interfaz gráfica y le devuelve la información visual. Mantiene una transferencia de datos continua con el motor de simulación hídrica y ordena la lectura o escritura de flujos de datos en el almacenamiento.

3° Módulo Motor de Simulación Hídrica: Mantiene una transferencia de datos directa con el gestor de inventario. Es el encargado de la emisión de alertas de sequía hacia la interfaz gráfica y del envío de estado actual del sistema hacia el bot de mensajería.

4° Módulo Bot de Telegram Recibe el envío de estado directamente desde el motor de simulación hídrica, sirviendo como un método de notificación externa además de un monitoreo.

5° Módulo de Persistencia Local Funciona como componente de almacenamiento, dedicándose solo a la lectura y escritura de flujos ordenados por el gestor de inventario.

Tabla 1: Módulos de la arquitectura del sistema

Módulo	**Propósito**
Interfaz Gráfica (GUI)	Muestra la interfaz Qt (RNF1) para registrar especímenes (RF1), visualizar el inventario con fotos y barras QProgressBaar(RF2), y activar el riego (RF4).
Gestor de Inventario (INV)	Administra la lógica POO de objetos planta y habitacion asignados en el *heap*, conectando la GUI con la simulación y la persistencia.
Motor de Simulación Hídrica (MSH)	Calcula la evaporación diaria según la especie y la orientación solar de la ventana, además de emitir alertas de sequía(RNF3) (RF3)
Módulo de Persistencia local (PL)	Se encarga de la lectura y escritura automática (RF5) en archivos locales mediante flujos para inventario , historiales de simulaciones y rutas relativas de fotografías offline(RNF2)
Bot de Telegram (BDT)	Extensión asincrónica (RNF4) que envía notificaciones de alertas remotas al móvil cuando existe conexión a la red.

Matriz de Requisitos Funcionales y Módulos

Reescriba los requisitos funcionales, con su abreviatura. La siguiente matriz muestra qué módulos del sistema implementan qué requisitos funcionales.

[*NOTA*: Una línea vacía en la matriz indica un requisito funcional no implementado; en ese caso es probable que falte un módulo. Una columna vacía indica que un módulo no es útil para estos requisitos funcionales; en ese caso es posible que falte un requisito funcional. Una columna demasiado llena indica un módulo que participa en demasiados requisitos; esto podría ser síntoma de una mala modularización, porque un cambio en el módulo puede afectar a todos los requisitos. Algo similar ocurre con una fila demasiado llena, que corresponde a un requisito que se implementa en todas partes; un cambio en el requisito podría implicar cambios en todas los módulos.]

[*NOTA*: La matriz de abajo debe re-formatearse de modo que tenga columnas para todas los módulos y filas para todos los requisitos funcionales. Una vez re-formateada la matriz, ponga

una X en una celda para indicar que el módulo de la columna respectiva ayuda a implementar el requisito funcional de la fila respectiva.]

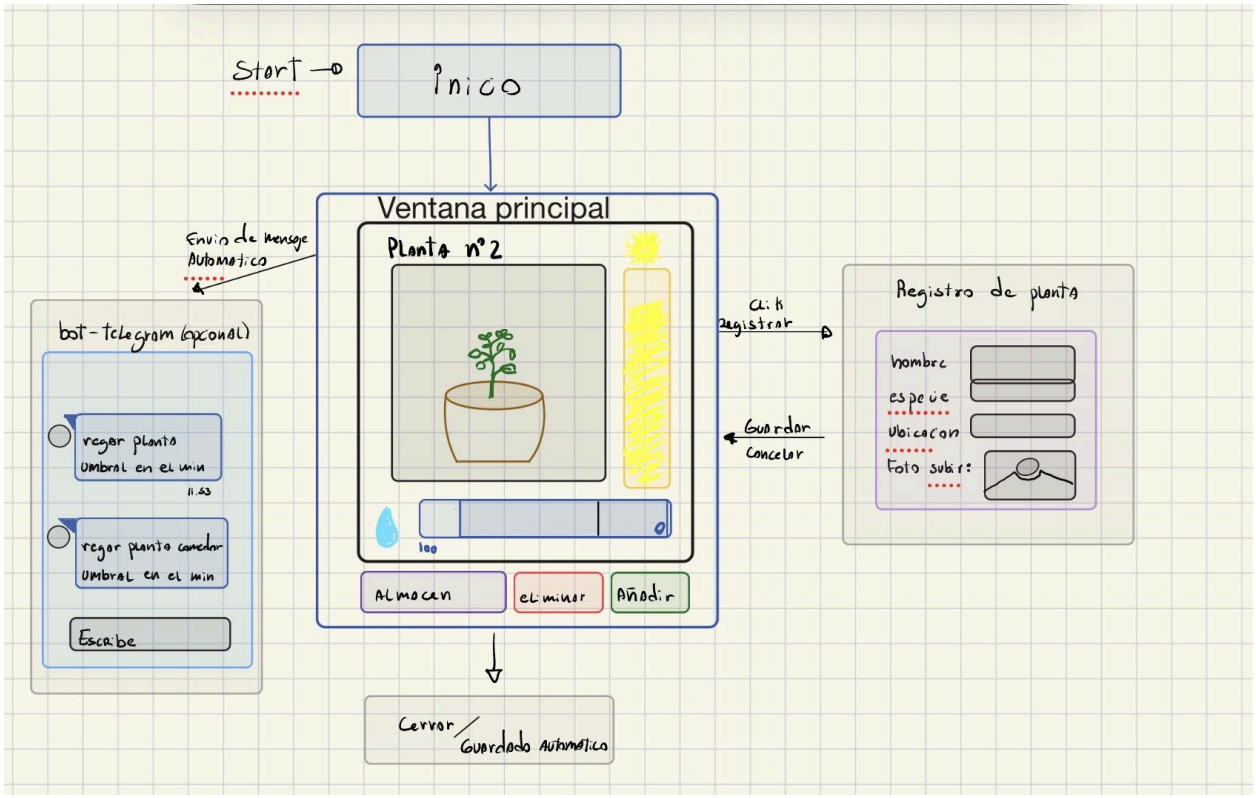
Tabla 2: Matriz de requisitos funcionales y módulos

	GUI	INV	MSH	PL	BDT					
RF1	X	X								
RF2	X	X								
RF3			X		X					
RF4	X	X								
RF5		X		X						
RF6										
RF7										
RF8										

Diseño de Interfaces

Modelo de Navegación

Diagrama general que muestra las interacciones (eventos o acciones) entre los elementos de interfaces usuarias del sistema. Ejemplos (nótese que en ambos modelos el punto de partida es claro):



Prototipos de Interfaces Usuarias

[Repetir para las GUI principales del sistema]

Prototipo [XX]

[Mostrar gráficamente el diseño de la interfaz (no funcional). El nombre referenciado debe ser coherente con el nombre correspondiente especificado en 3.1. Detallar los eventos que se pueden gatillar desde la interfaz, completando la siguiente tabla:]

Evento	Interacción	Acción	Objeto afectado
[Nombre del evento]	[Descripción de acción que gatilla el evento (e.g apretar un botón, tocar pantalla...).]	[Descripción de la acción resultante del evento. (e.g. Se abrió ventana, se muestra otro elemento, mensaje)]	[Objeto sobre el cual se ejerce la acción. Ser coherente con los nombres especificados en 3.1 y en la visualización gráfica de la interfaz].

Click de registro	Hacer click en el botón Añadir	Abre la pantalla de registro de un nuevo espécimen	Abre la pantalla de registro de un nuevo espécimen.
Ver almacén	Hacer click en el botón Almacen	Muestra la lista/almacén completo de plantas registradas	ventana principal/almazen
Eliminar planta	Hacer click en el botón Eliminar	Quita la planta seleccionada del inventario	Quita la planta seleccionada del inventario
Enviar mensaje automático	Activación automática cuando la humedad cae bajo el umbral mínimo.	Envía un mensaje de alerta automático con el estado de riego.	bot telegram (opcional)
Cerrar	Salir o cerrar la aplicación.	Realiza el guardado automático del estado actual	Archivo de almacenamiento

Registro de plantas

Subir foto	Hacer click en el recuadro para subir foto	Abre la ventana del sistema para seleccionar e importar una imagen local	Campo de foto/registro planta
Guardar registro	Hacer click en la	Registra los datos	ventana

	opción guardar	ingresados, crea la planta y vuelve a la pantalla principal.	principal/objeto planta
Cancelar el registro	Hacer click en la opción cancelar	Descarta los cambios e interrumpe el registro volviendo a la pantalla principal.	ventana principal

Repositorio: https://github.com/lukastudillo/BioSpace